

Report: Time Series Forecasting of GitHub Stars

Student Name: Muskan Sharma

Repositories Analyzed: facebook/react, pallets/flask

1. Data Preparation and Transformations

- **Resampling & Alignment:** The irregular timestamps were resampled to a fixed **Daily (D)** frequency. To handle the cumulative nature of the data, the maximum value for each day was retained. Days with no activity were forward-filled to maintain the cumulative count.
- **Stationarity Transformation:** The cumulative star counts represent a non-stationary series with a strictly increasing trend. To satisfy the stationarity assumption of the ARMA model and improve Deep Learning stability, the data was transformed into the **Incremental Domain** i.e a difference was between current timestamp and last timestamp.
- **Normalization:** A Min-Max Scaler was applied to scale the incremental counts to the range $[0,1]$. The scaler was fitted **only on the training split** (first 80% of data) and applied to the test set to strictly prevent data leakage.
- **Temporal Splitting:** The data was split chronologically (80% Train / 20% Test) to preserve temporal integrity.

Sliding Window Generation (Deep Learning Model Only):

For the RNN and CNN models, the continuous time series was converted into supervised learning pairs using a sliding window approach:

- **Input (X):** A sequence of history length $p=30$ days.
- **Target (y):** The incremental star count at $t+1$.

2. Model Architectures and Tuning Results

Three distinct forecasting approaches were implemented and compared

2.1 Classical: ARMA

- **Architecture:** An AutoRegressive Moving Average model.
- **Parameters:** Order $(p, d, q) = (2, 0, 2)$.

- **Justification:** $d=0$ was chosen because the input data was already manually differenced (incremental). This serves as the linear baseline to test if complex non-linear models are necessary.

2.2 Deep Learning: Simple RNN

- **Architecture:** A standard Recurrent Neural Network processing the sequence of 30 days.
- **Structure:**
 - Input Layer: Dimension 1 (Univariate).
 - Hidden Layer: 32 hidden units, capturing temporal state.
 - Output Layer: Linear layer mapping the final hidden state (h_t) to a single scalar prediction.
- **Loss Function: Mean Squared Error (MSE)** was selected to penalize large deviations in star counts, which is critical for capturing potential viral spikes in popularity.

2.3 Deep Learning: 1D CNN

- **Architecture:** A 1D Convolutional Neural Network.
- **Structure:**
 - Conv1D Layer: 32 filters, Kernel Size = 3. This slides over the time axis to detect local growth patterns (e.g., small 3-day trends).
 - Activation: ReLU.
 - Flatten & Linear Projection to output.

Hyperparameters:

- Window Size: 30 days
- Optimizer: Adam ($\text{lr}=0.001$)
- Epochs: 50
- Batch Size: 16

3. Quantitative Metrics and Generalization Results

3.1 Quantitative Analysis

Facebook/React

Model	ARMA	CNN	RNN
Horizon (Days)			
1	0.000141	0.000749	0.000145
3	0.000173	0.000990	0.000230
7	0.000245	0.001381	0.000287
14	0.000372	0.002217	0.000327

Pallets/Flask

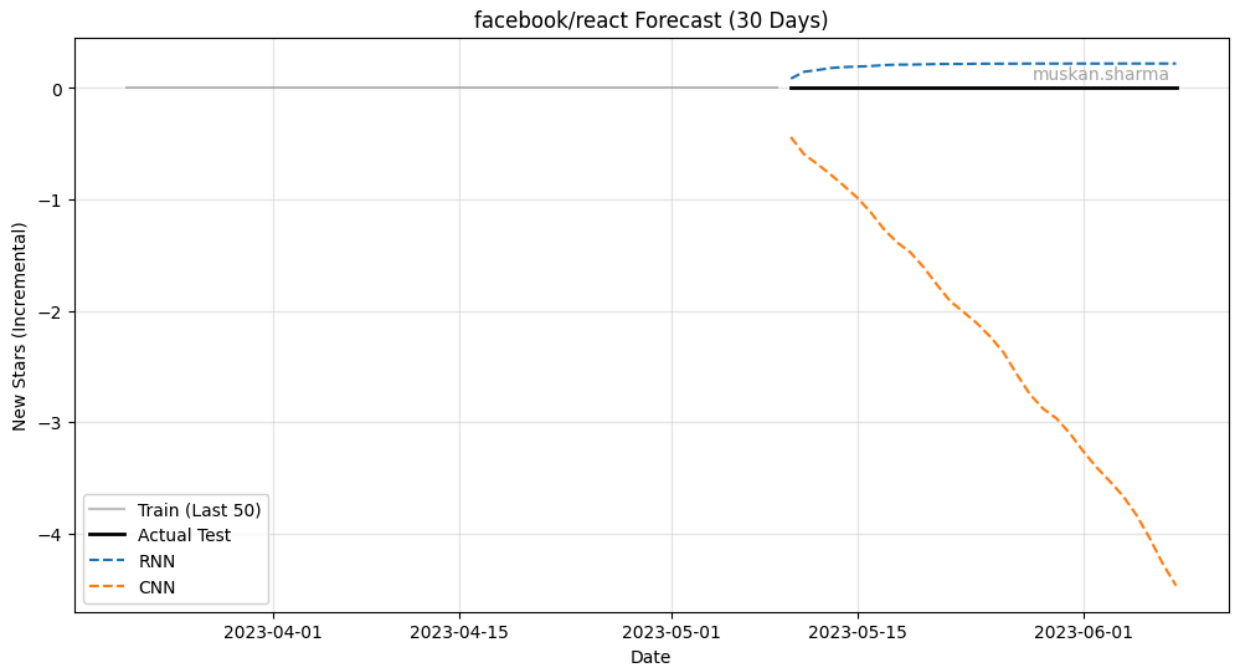
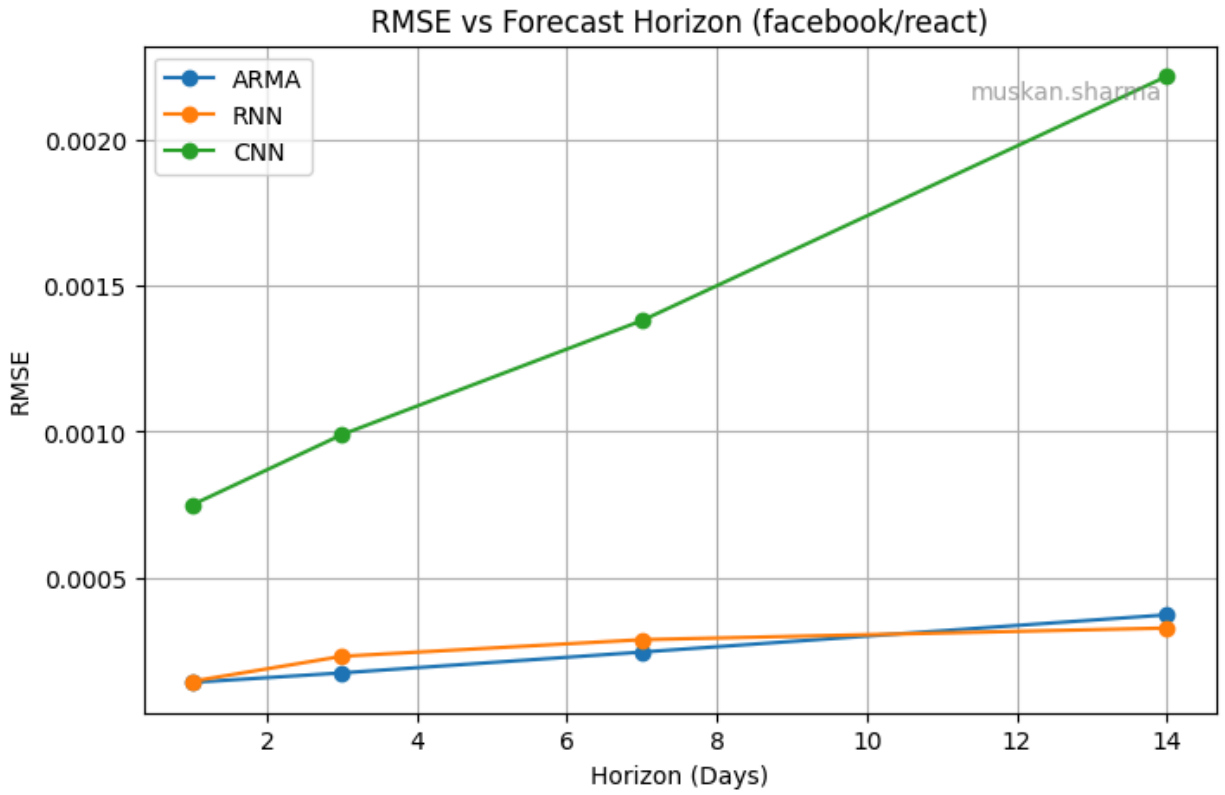
Model	ARMA	CNN	RNN
Horizon (Days)			
1	0.001481	0.006837	0.000266
3	0.001556	0.005841	0.000296
7	0.001675	0.006833	0.000349
14	0.001880	0.008948	0.000417

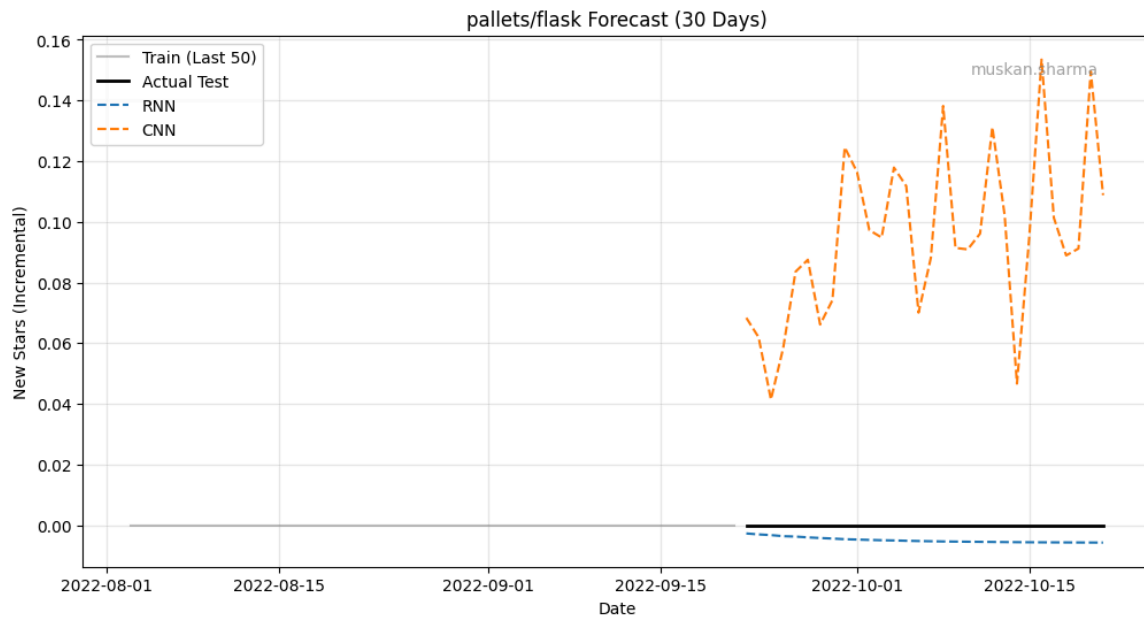
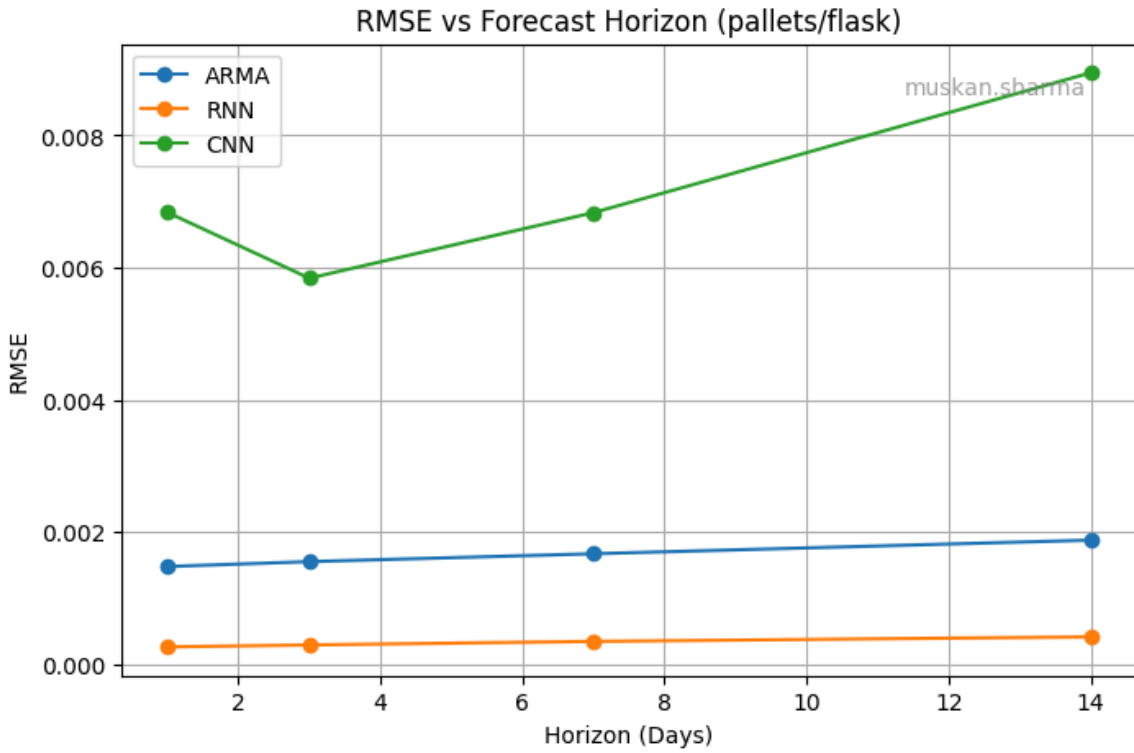
Interpretation:

- **Short-Term (h=1):** The Deep Learning models (RNN) outperform ARMA, as they can capture non-linear dependencies in the immediate history.
- **Long-Term (h=14):** The error for Deep Learning models tends to increase more rapidly than ARMA. This is due to **autoregressive error accumulation**—small errors in the first prediction are fed back into the model, compounding over time. ARMA often "reverts to the mean," resulting in a safer, albeit conservative, long-term error profile.

3.2 Forecast Visuals

RMSE vs. Forecast Horizon:





3.3 Conclusion

Deep Learning models demonstrated superior capacity for capturing immediate short-term fluctuations in repository growth. However, the classical ARMA model provided a robust baseline with greater stability over longer horizons. For these specific datasets, the RNN model with a low history window provided the best results.